University of Pittsburgh

CS 1541 Introduction

Instructor Introduction
Course Introduction

Wonsun Ahn
Department of Computer Science
School of Computing and Information

Instructor Introduction

My Technical Background

- Wonsun Ahn
  - First name is pronounced *one-sun* (if you can manage)
  - Or you can just call me Dr. Ahn (rhymes with *naan*)
- PhD in CPU Design and Compilers
  - University of Illinois at Urbana Champaign
- Industry Experience
  - Bluebird Corporation (70-person startup company)
    - Manufactures industrial hand-held devices
    - Me: Built software stack based on Windows Embedded
  - IBM Research (thousands of people)
    - Does next-gen stuff like carbon nanotubes, quantum computers
    - Me: Designed supercomputers for ease of parallel programming

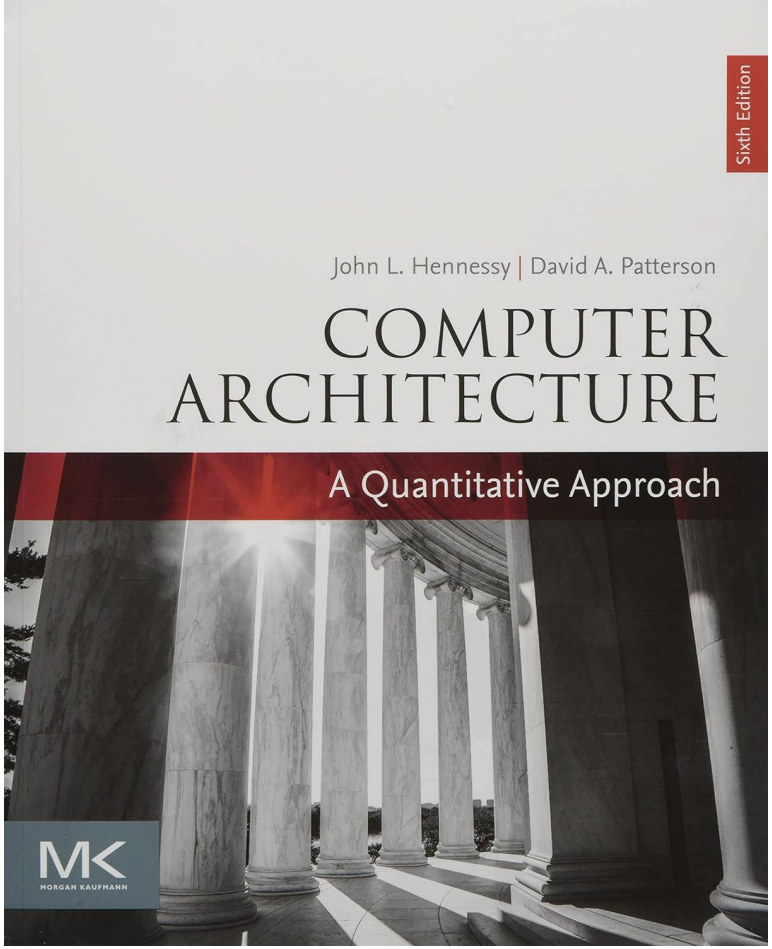
Course Introduction

Structure of the Course

- (40% of grade) Midterm and final exam
- (20% of grade) Project
  - Implementing a CPU simulator using C/C++ programming language
- (20% of grade) Four written homeworks, 5% each
- (10% of grade) Two paper reading homeworks, 5% each
- (10% of grade) Participation
  - Attendance, In-class / out-of-class participation
- Class resources:
  - GitHub: syllabus, lectures, homeworks, projects
  - Tophat: attendance
  - GradeScope: homework / projects submission, grading and feedback
  - Microsoft Teams: all out-of-class communication

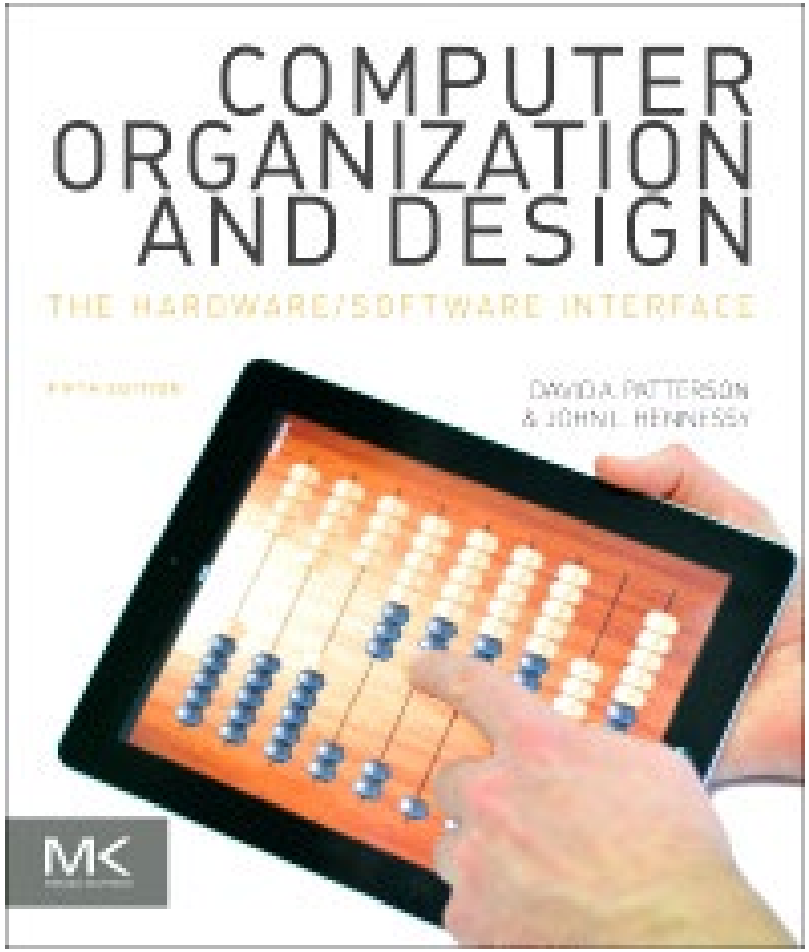
Required Textbook

- "Computer Architecture: A Quantitative Approach" by John Hennessy and David Patterson. Sixth Edition - Morgan & Kaufmann.



Optional Textbook (from CS 1541)

- "Computer Organization and Design - The Hardware/Software Interface" by David Patterson and John Hennessy Fifth Edition - Morgan & Kaufmann.



For More Details

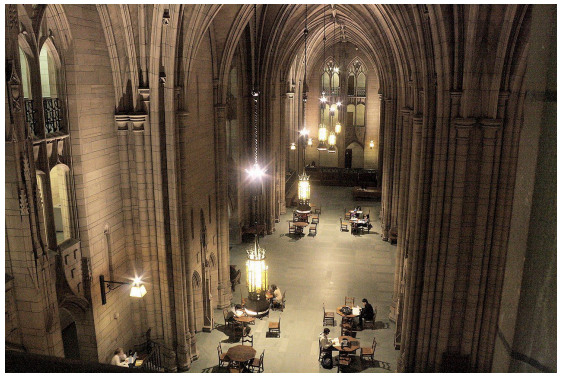
- Please refer to the course repository: https://github.com/wonsunahn/CS2410_Spring2026

- There you will find:
  - The syllabus
  - The course schedule
  - Lectures, assignments, and other course material (Will be posted as the semester progresses)

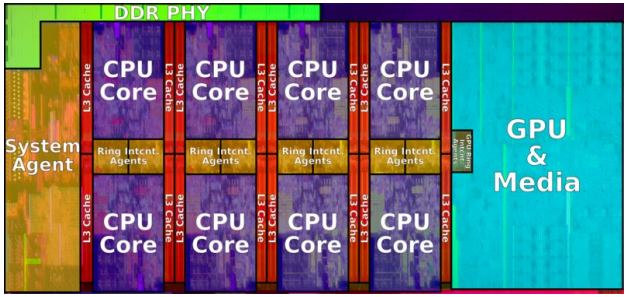
Computer Architecture. What is it?

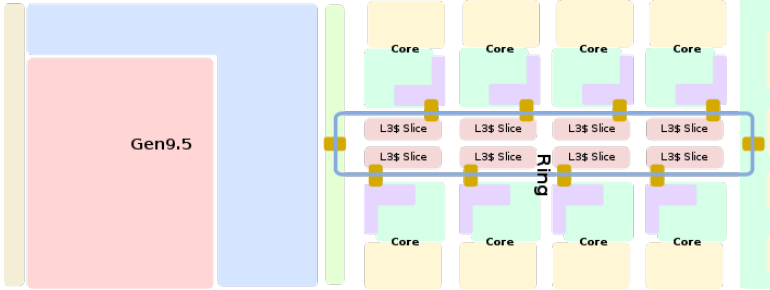
Computer Architecture: the architecture of a CPU

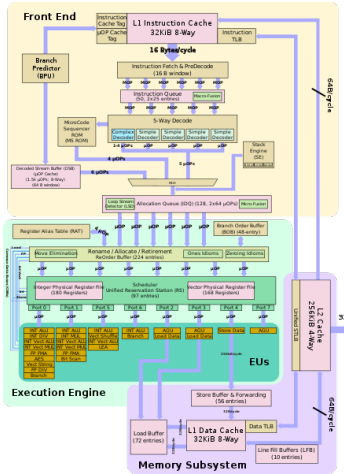
- Every building has an architecture:



- Every CPU has an architecture as well:



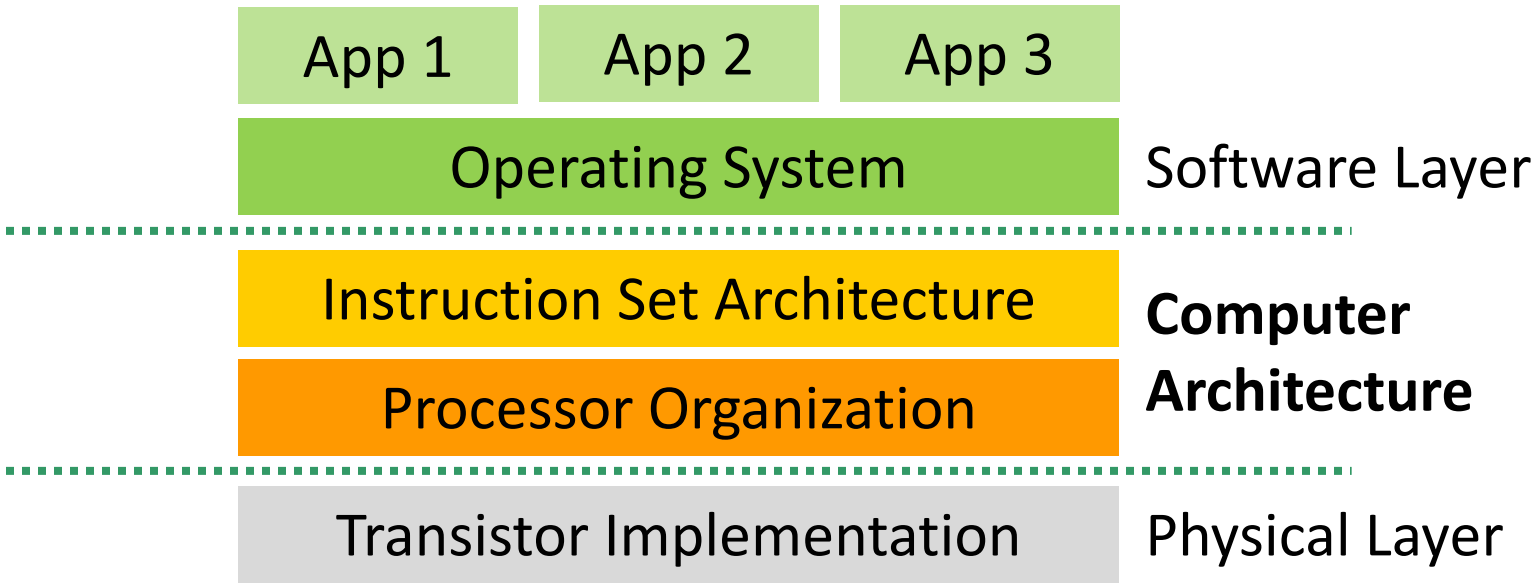




ISA: the architecture of an instruction set

- At a high-level: how a computer is built
  - Computer here meaning the processor (CPU)
- You probably heard of a similar term before: ISA
  - ISA (Instruction Set Architecture)
- Review: what is defined by an ISA?
  - *Set of instructions* usable by the computer
  - *Set of registers* available in the computer
  - Other functional attributes
- What is *not* defined by an ISA?
  - *Speed* of computer
  - *Energy efficiency* of computer
  - *Reliability* of computer
  - Other performance attributes

Computer Architecture = ISA+Processor Organization



- Computer Architecture = ISA + Processor Organization
  - Processor organization is also called *Microarchitecture*
- Given an ISA, performance is determined by:
  - Processor organization (internal design of the processor)
  - Transistor implementation (semiconductor technology)

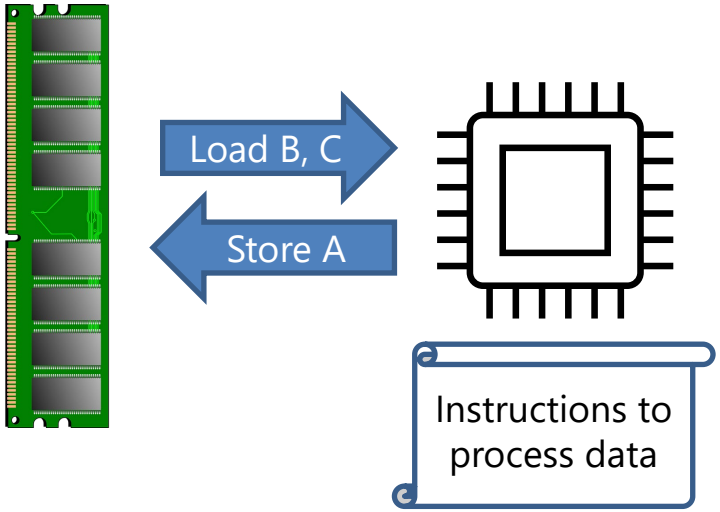
A CPU is like a factory

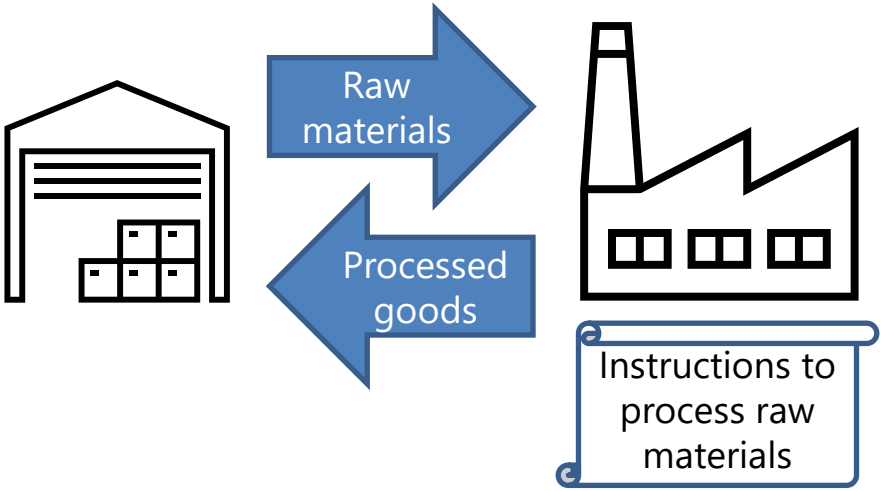
- A program written in Python, Java, or C is converted to instructions

```
A = B + C;
```

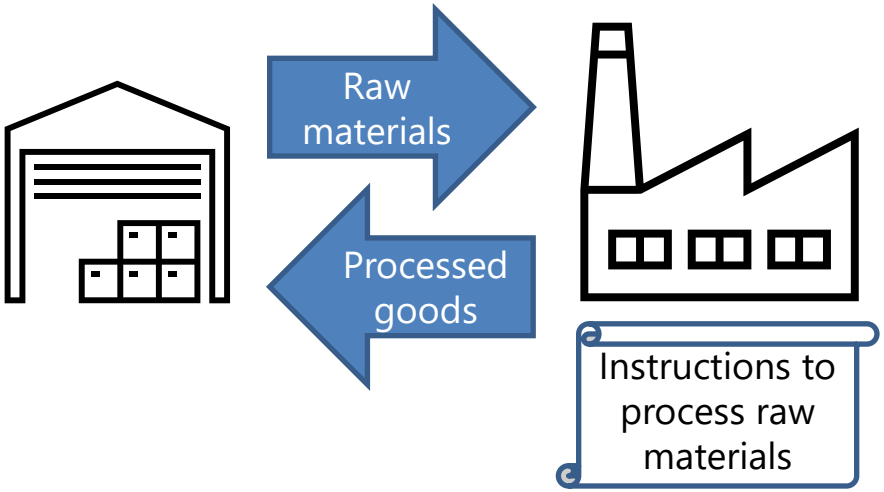
```
LOAD B, R1
LOAD C, R2
ADD R3, R1, R2
STORE A, R3
```

- A CPU processes data according to instructions, just like a factory





Factory efficiency depends on factory design

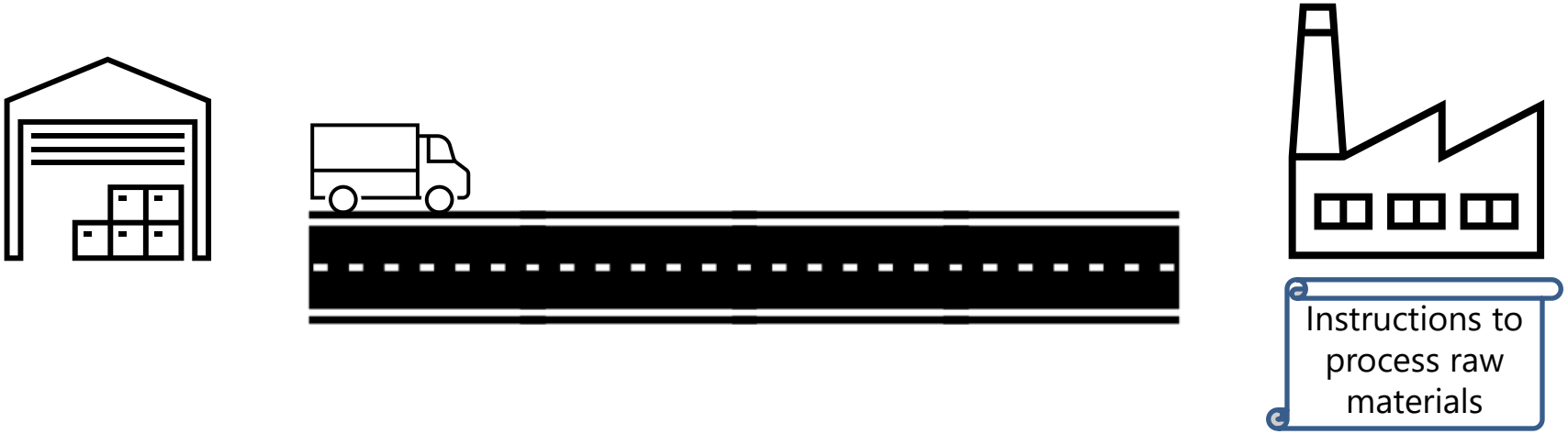


1. How efficiently goods are moved between warehouse and factory
2. How efficiently workers process goods in factory

How to design a shirt factory
Part 1: Logistics

University of Pittsburgh

You cannot succeed without good logistics



- If you can't move goods in a timely manner your output will suffer
- Factory efficiency doesn't matter if logistics is the bottleneck
  - If deliveries are slow (high memory latency)
  - Or do not arrive in sufficient quantities (low memory bandwidth)

Latency vs. Bandwidth

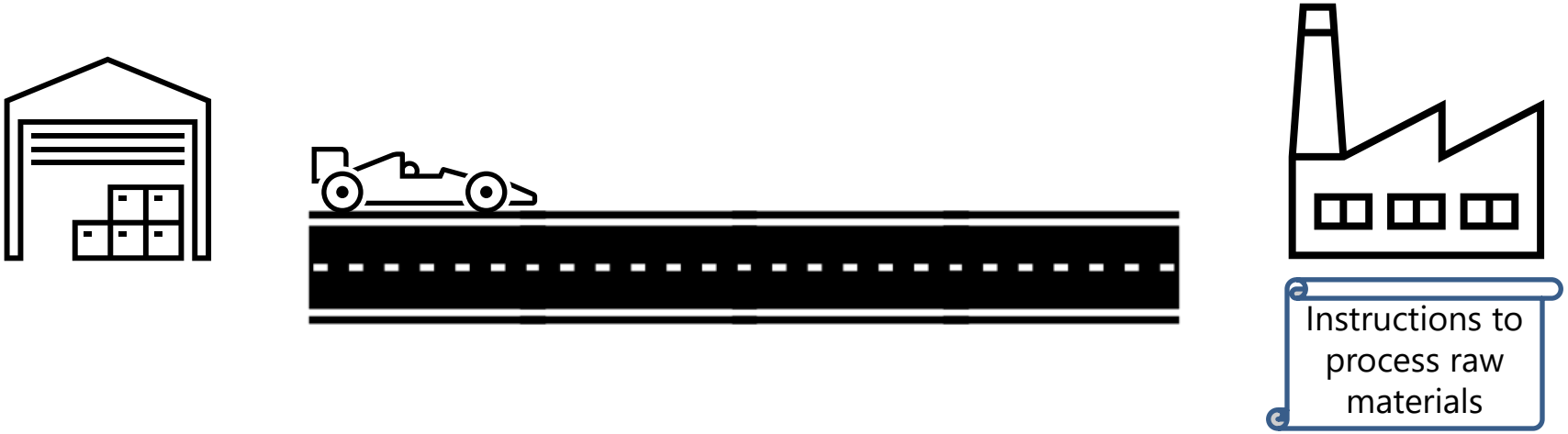
- Performance of a transfer system can be measured in two ways:
  - **Latency**: hours to handle a single delivery

vs.

  - **Bandwidth**: maximum volume handled per hour

vs.

Memory latency does not put a cap on performance

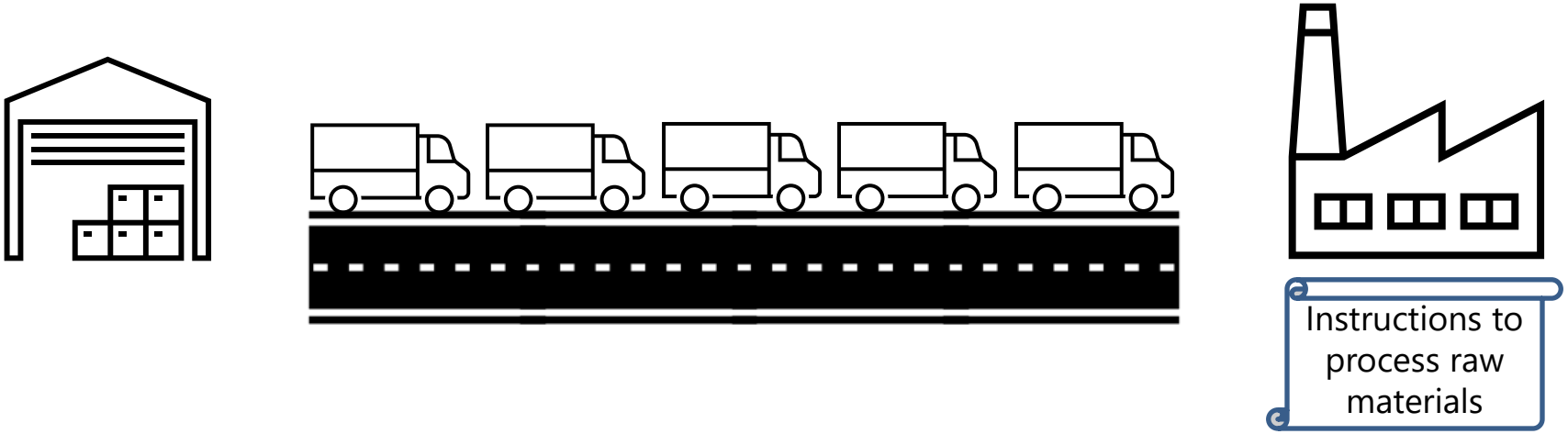


- Race cars are rarely used for logistics. Trucks are. Why?

1. Can place orders ahead of time to arrive just in time (*prefetching*)
2. Factory can produce something else while waiting (*scheduling*)

- Latency can be circumvented without impacting performance.
  - Except when neither prefetching or scheduling works

Memory bandwidth does put a cap on performance



- If the road is fully utilized but still can't keep up with demand…
  - Road handles 5 trucks / minute but factory needs 50 trucks / minute

- No amount of pre-ordering or scheduling can alleviate this problem

→ Unlike latency, bandwidth puts a hard limit on performance!

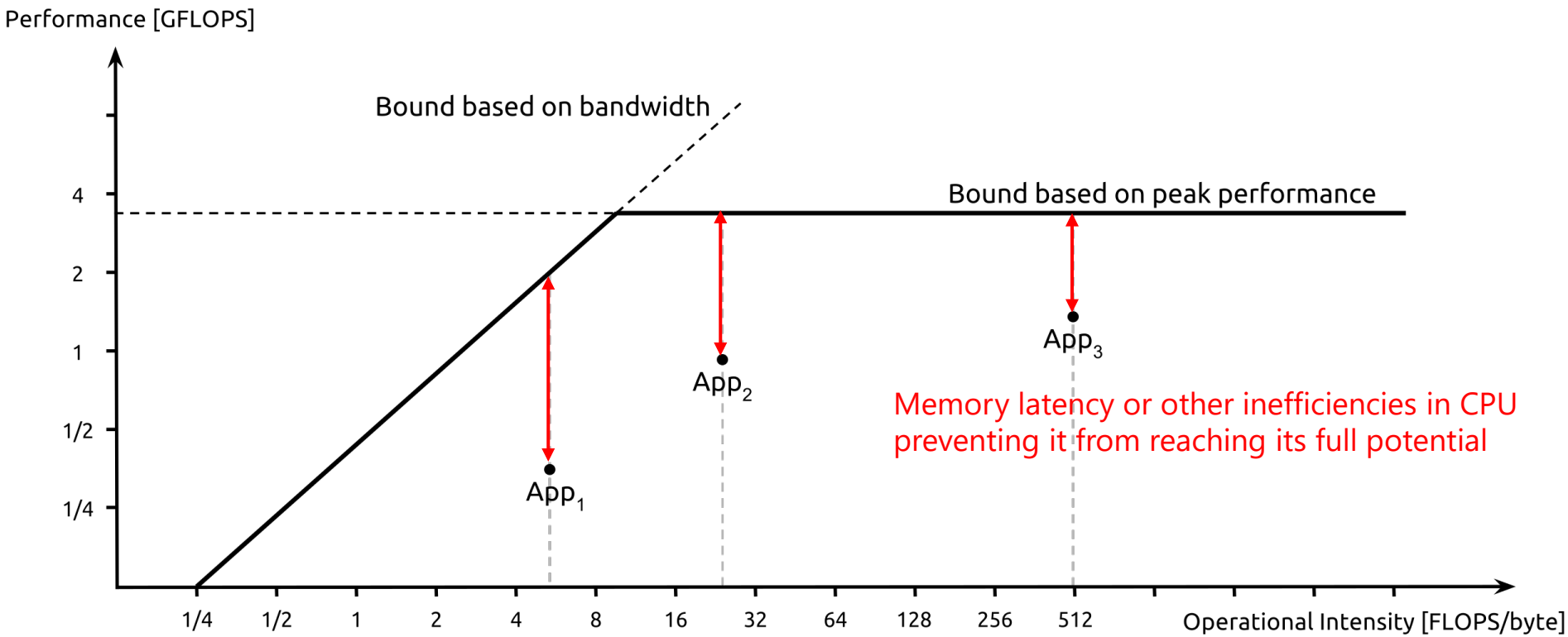
Memory bandwidth does put a cap on performance

- When you get a traffic jam, performance is dictated by bandwidth
  - How quickly you pull data in, rather than how fast you process it
  - **Operational intensity** = work (FLOP) per memory access (byte)
  - Performance = work / second
    = work / byte * byte / second
    = operational intensity * memory bandwidth

→ Linear relationship between performance and operational intensity

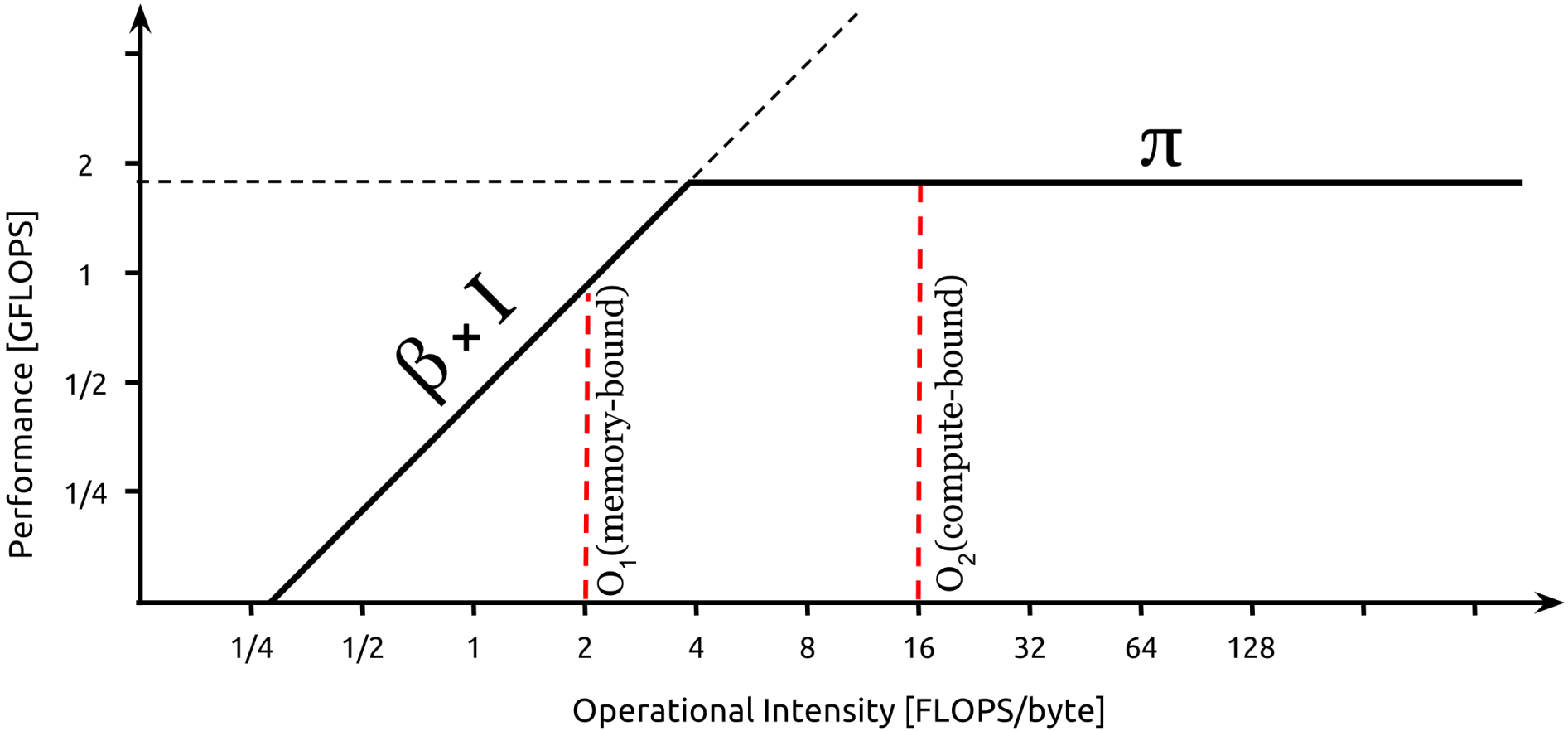
The Roofline Model: A bird's eye view of performance

- **Roofline**: theoretical performance achievable given an intensity
  - Formed by memory bandwidth bound + peak CPU performance



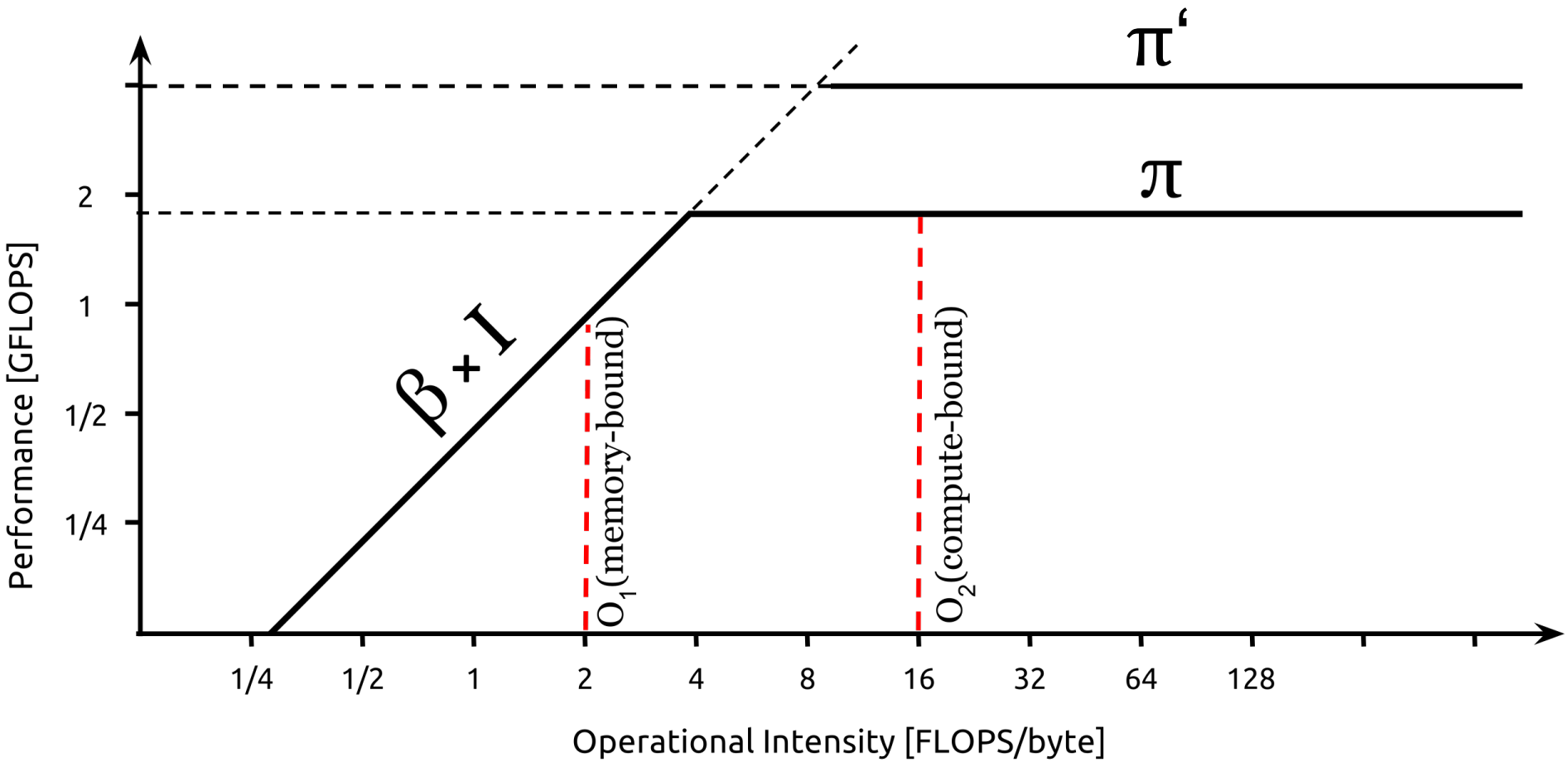
Memory bound vs. compute bound apps

- I = intensity, β = peak bandwidth, π = peak performance



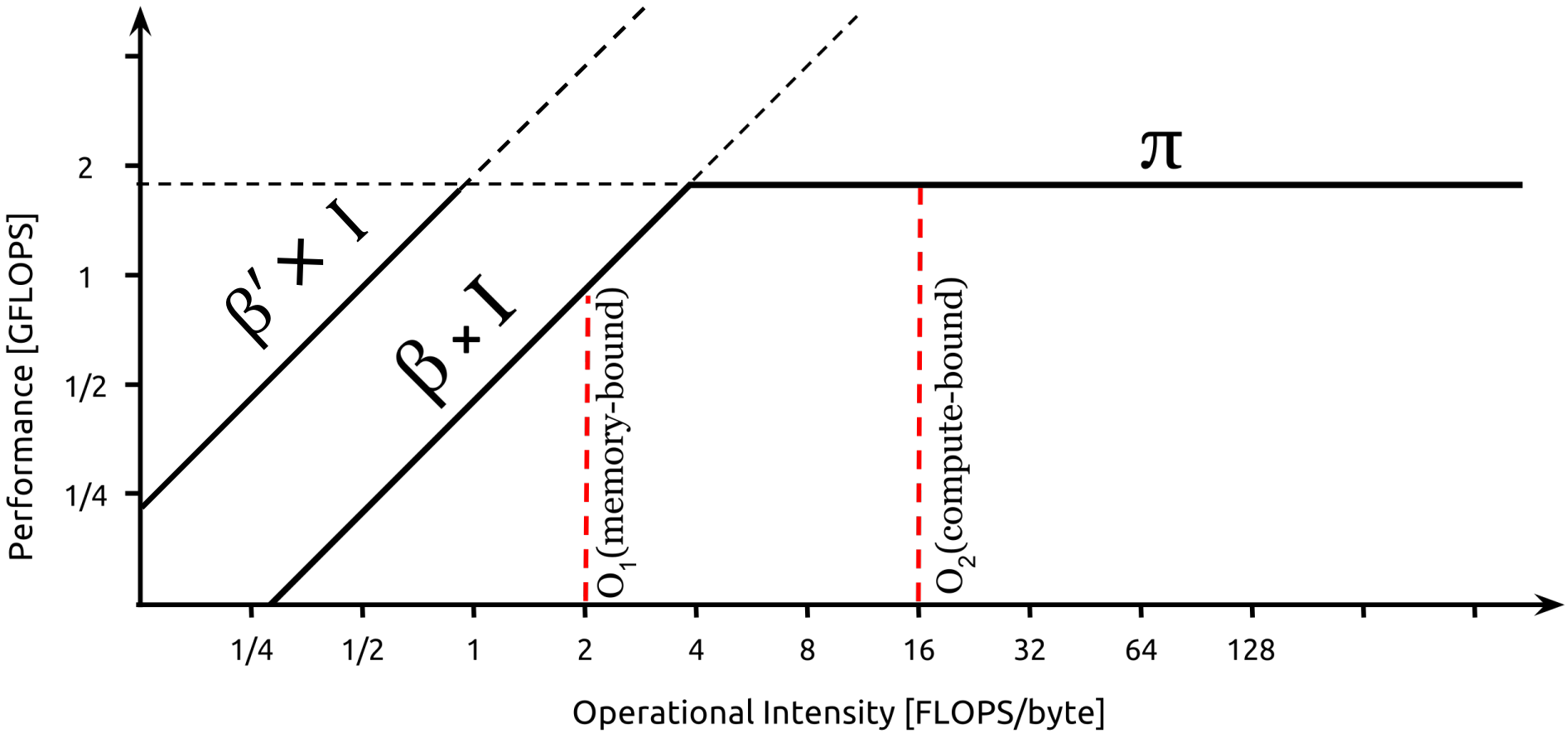
Peak performance increase makes more apps memory bound

- Causes inflection point to shift to the right
  → Now more applications are memory bound.

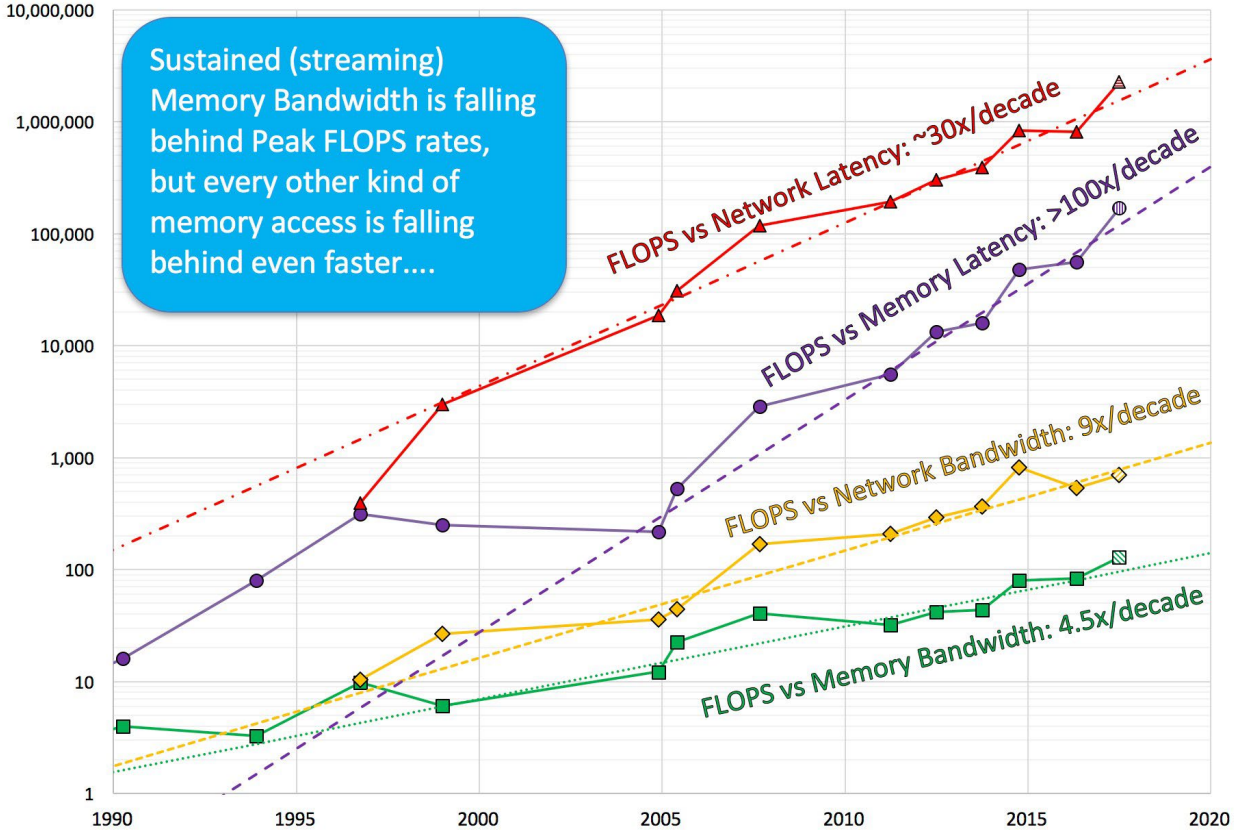


Bandwidth increase makes more apps compute bound

- log (Performance) = log (β × I) = log (β) + log (I)
  → Increase in β shifts the ceiling up, not change its slope.



Memory Wall: widening gap between memory and CPU

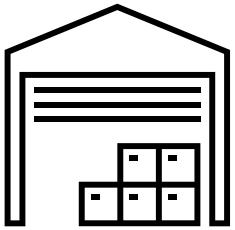


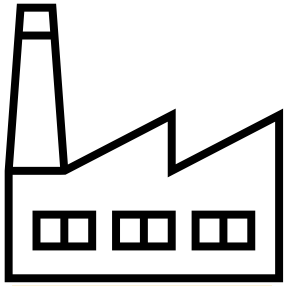
Source: SC16 Invited Talk ""Memory Bandwidth and System Balance in HPC Systems" by John D. McCalpin

- FLOPS = floating point operations per second (CPU speed)

University of Pittsburgh

Cache: storage in CPU for frequently accessed data



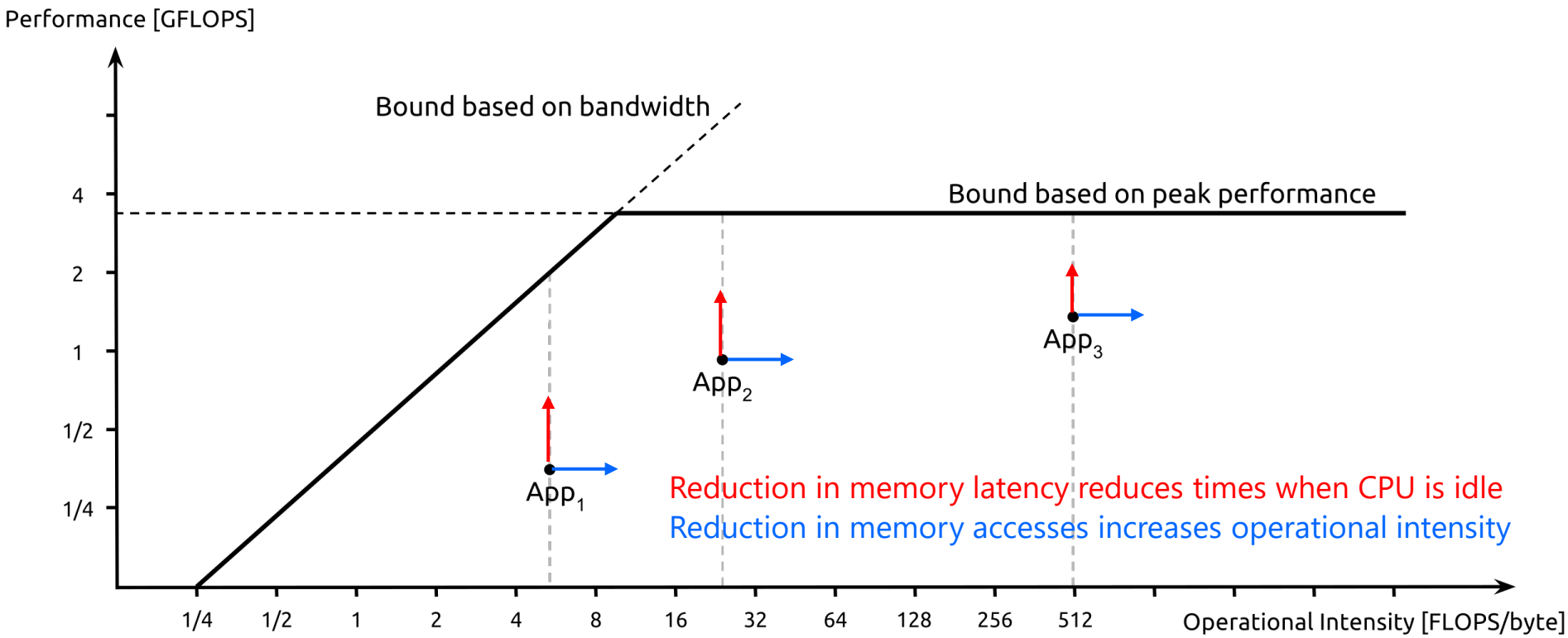


- What if we had a storage **cache** in factory premises?
  - Store frequently used inventory nearby

- Then, both memory latency and bandwidth problems are alleviated
  - Latency: access is quick when material is found in the cache
  - Bandwidth: less need to go back and forth to the warehouse

Effect of Cache on Roofline Model

- With caching, apps shift upwards and rightwards



Caches have become bigger and bigger

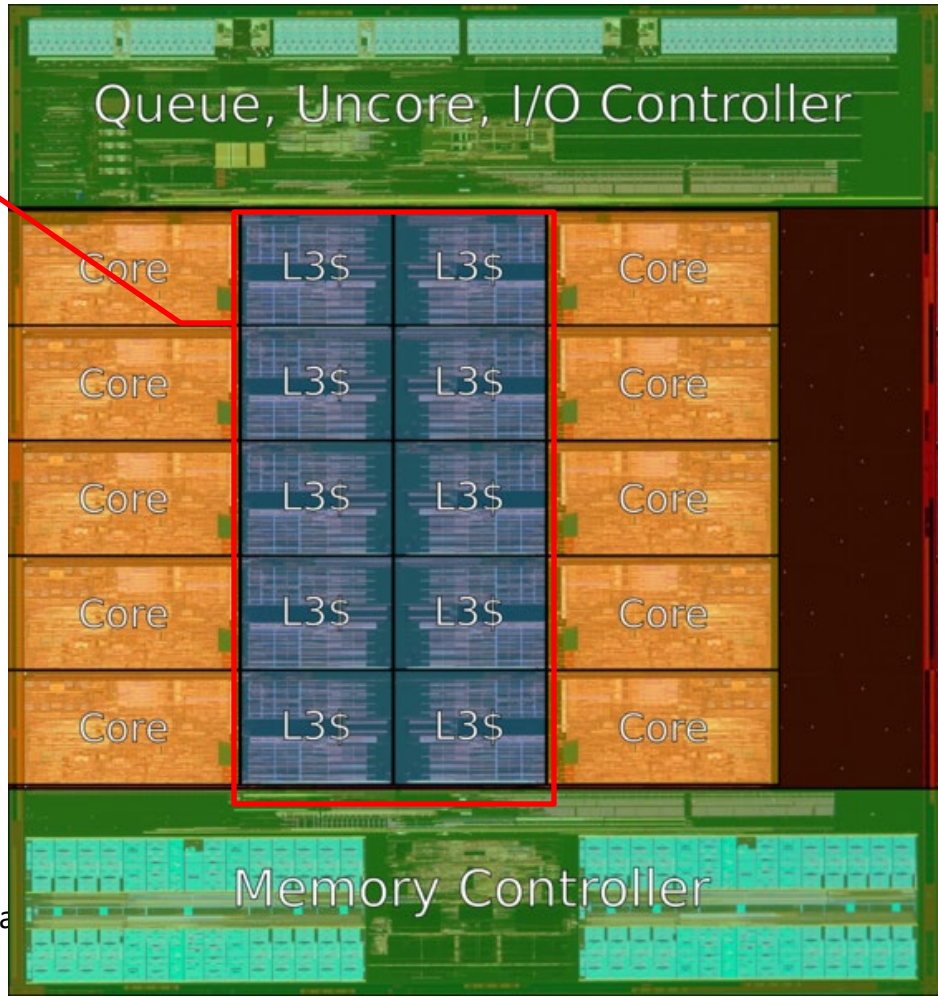
- Intel Xeon Broadwell CPU

Caches

- Caches take up almost as much real estate as cores!

- Memory latency: 80 ns
  Memory bandwidth: 100 GB/s

- Cache latency: 1~40 ns
  Cache bandwidth: 400 GB/s ~1 TB/s

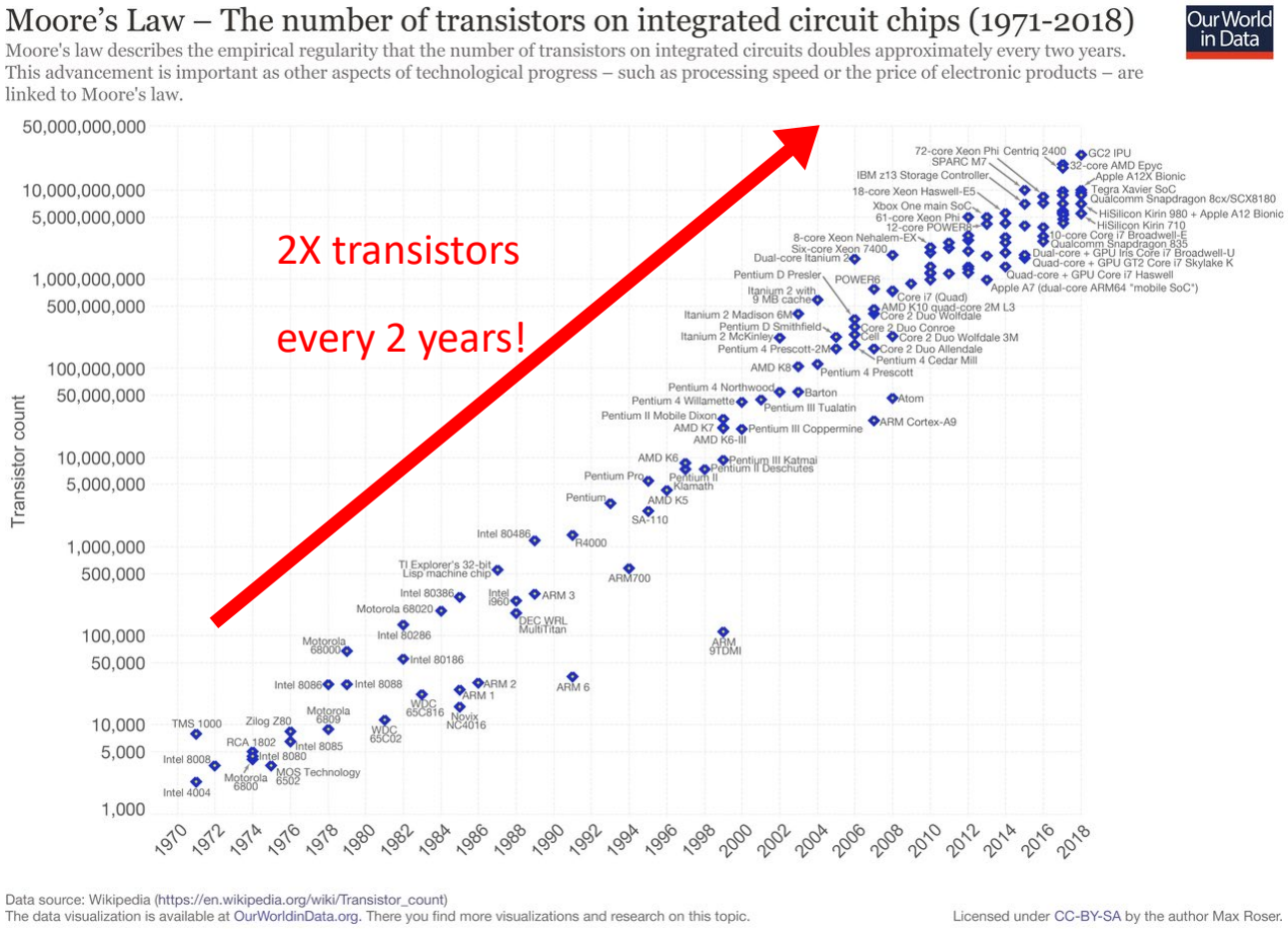


Ref:
https://www.intel.com/content/www/us/en/developer/articles/technica
/memory-performance-in-a-nutshell.html

How to design a shirt factory Part 2: Worker Utilization

University of Pittsburgh

Moore’s Law

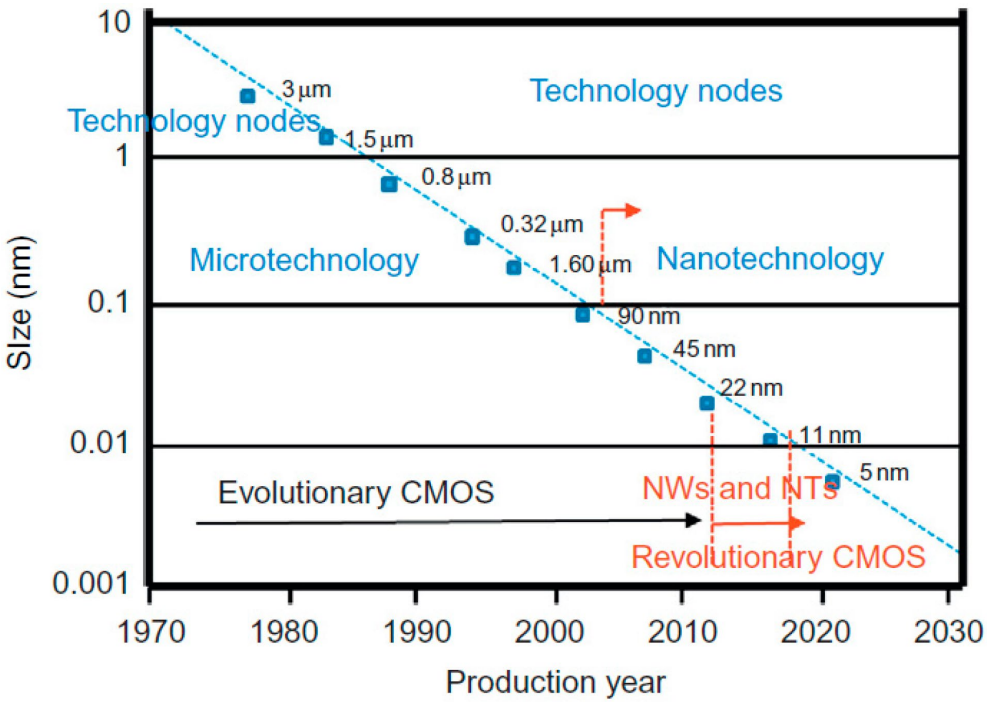


Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

University of Pittsburgh

Miniaturization of Transistors



Data source: Radamson, H.H.; He, X.; Zhang, Q.; Liu, J.; Cui, H.; Xiang, J.; Kong, Z.; Xiong, W.; Li, J.; Gao, J.; Yang, H.; Gu, S.; Zhao, X.; Du, Y.; Yu, J.; Wang, G. Miniaturization of CMOS. *Micromachines* **2019**, *10*, 293.

- Moore's Law has been driven by transistor miniaturization

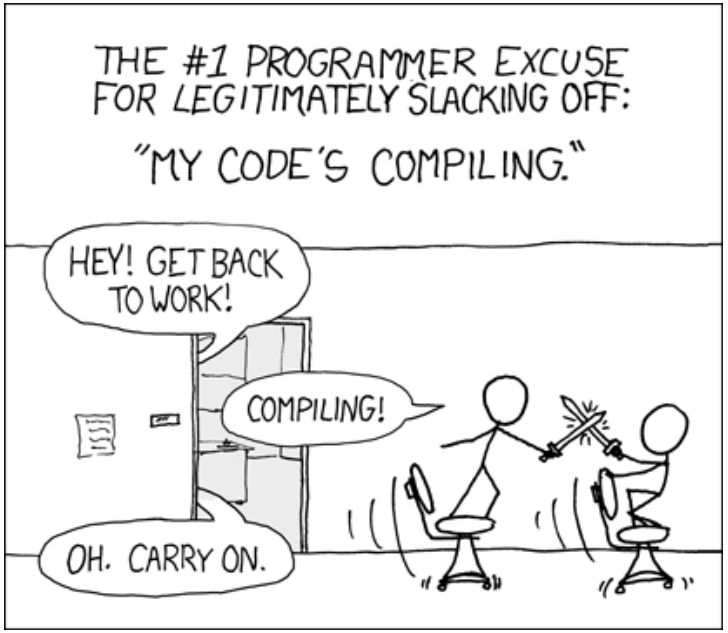
University of Pittsburgh

Our factory is bursting with workers!

- Transistors are cobbled together to create **Functional Units**
  - Things like Add Unit, Multiply Unit, Load Unit, Store Unit

- With billions of transistors, CPUs can contain many Functional Units

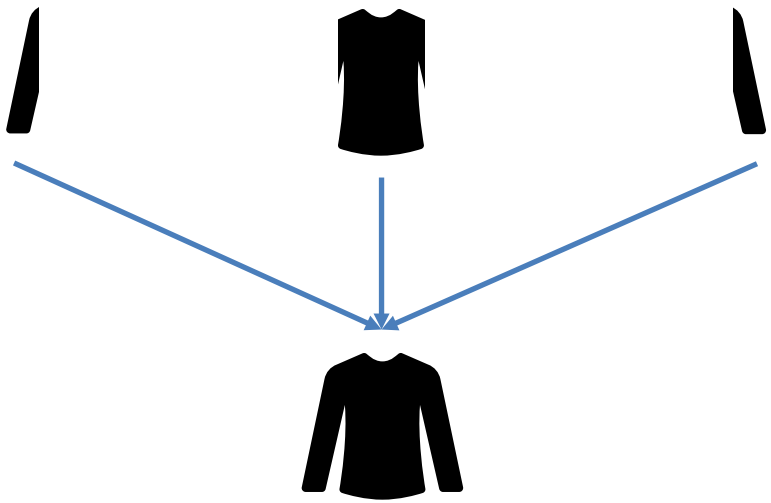
- What is the problem with a factory with a lot of workers?
  - It is hard to keep all the workers productive!
  - Workers find an excuse to slack (and often those excuses are valid)
  - Same goes for Functional Units

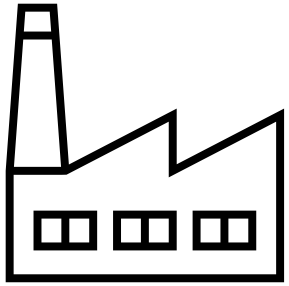


Shirt manufacturing instructions

- Let's assume one man hour for each instruction.
- How many workers would you hire?

- **Data Dependence Graph** for shirt manufacturing



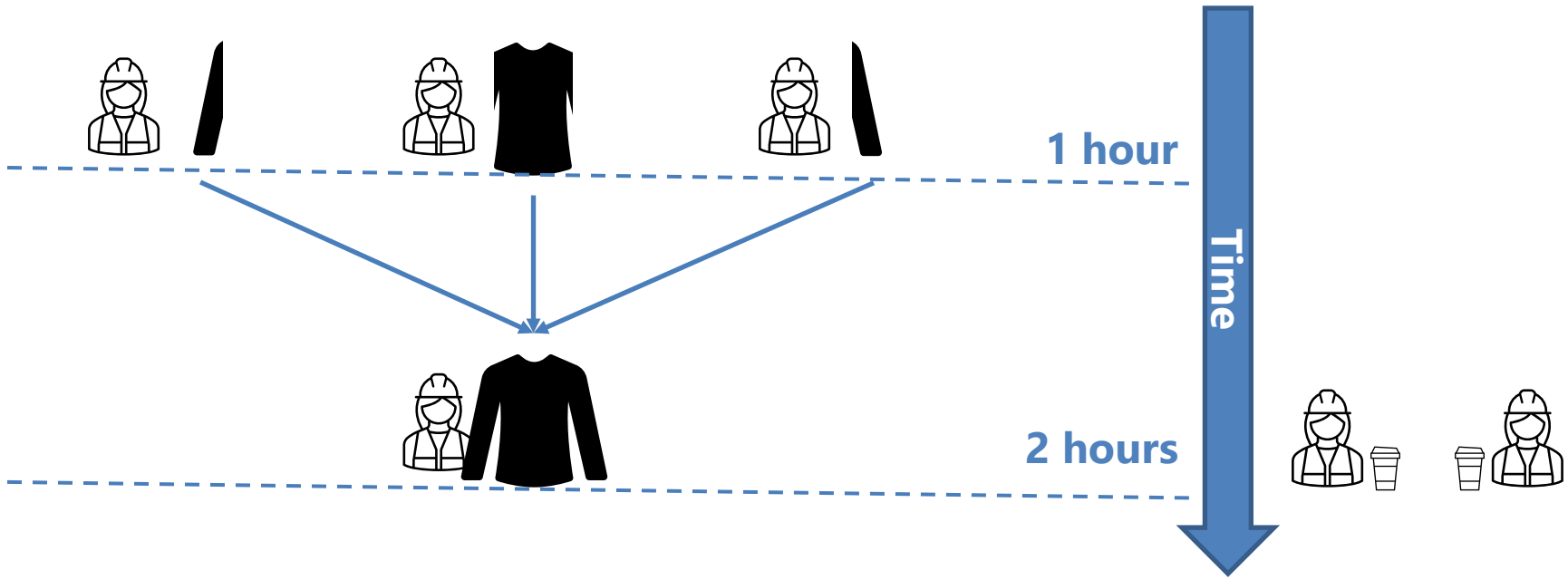


[Instructions]

1. Cut fabric for left sleeve
2. Cut fabric for right sleeve
3. Cut fabric for body piece
4. Stitch pieces together

3 workers = fastest but low utilization

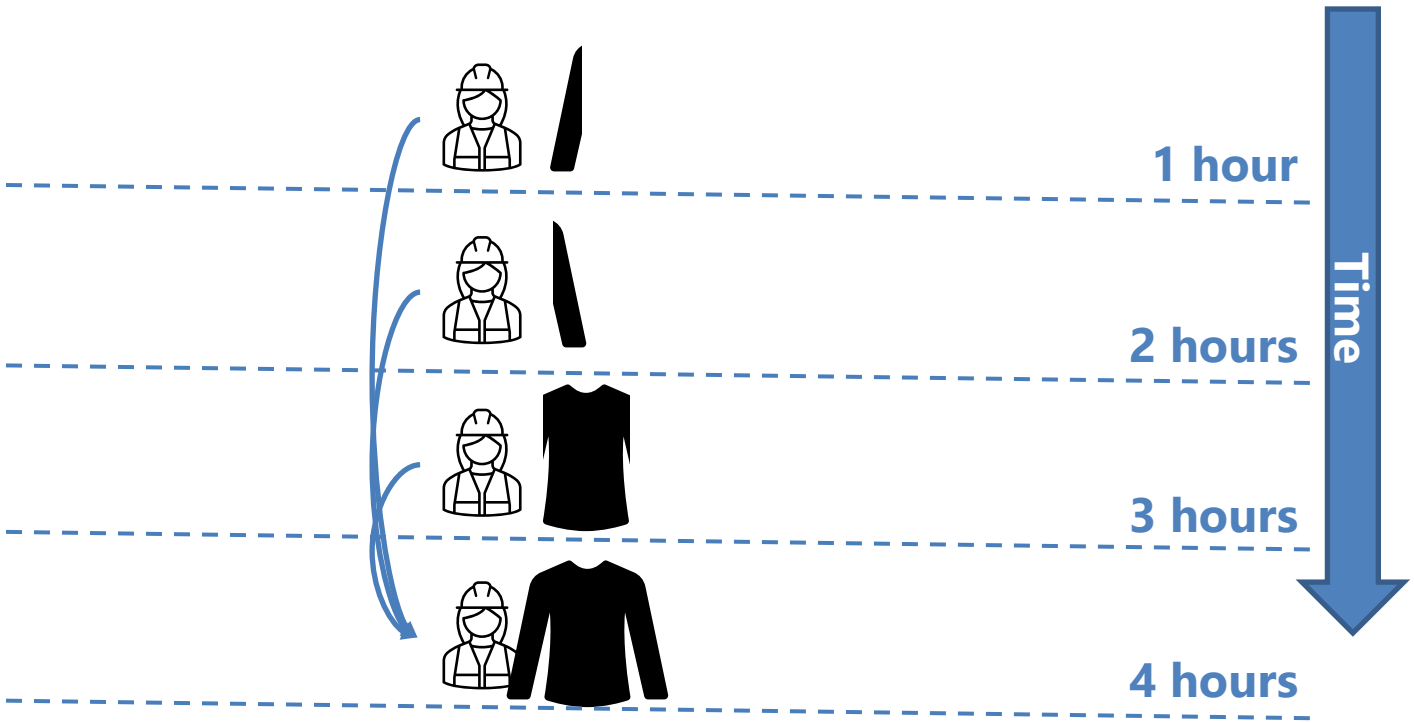
- Can produce a shirt in just 2 hours:



- During the 2nd hour, 2 workers were slacking over coffee. Oh no!
- Utilization = 4 steps / 6 worker-hours * 100 ≈ 67%

1 worker = slow but highest utilization

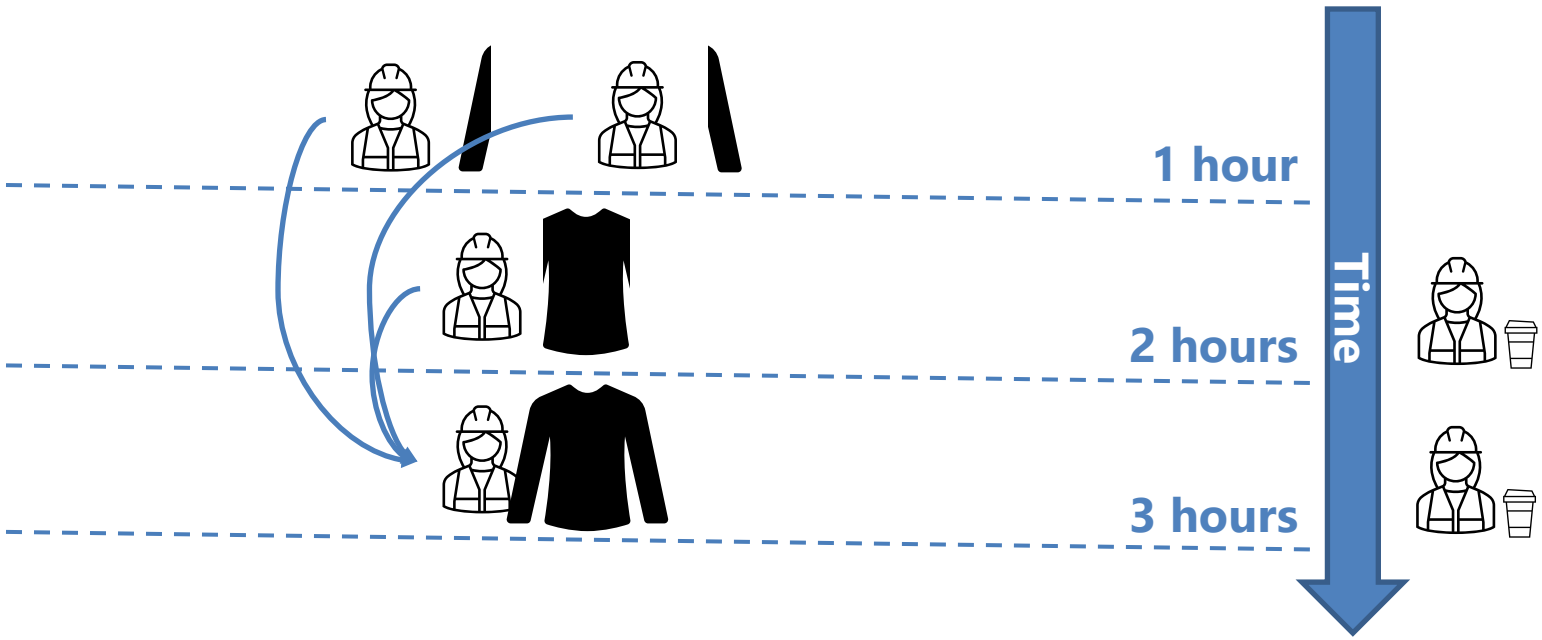
- Needs 4 hours to produce a shirt, but no slacking!



- Utilization = 4 steps / 4 worker-hours * 100 = 100%

2 workers = mediocre speed and low utilization

- Needs 3 hours with some slacking:



- Utilization = 4 steps / 6 worker-hours * 100 ≈ 67%
- Same utilization as 3 workers but just slower. Fail!

What are the implications of low utilization?

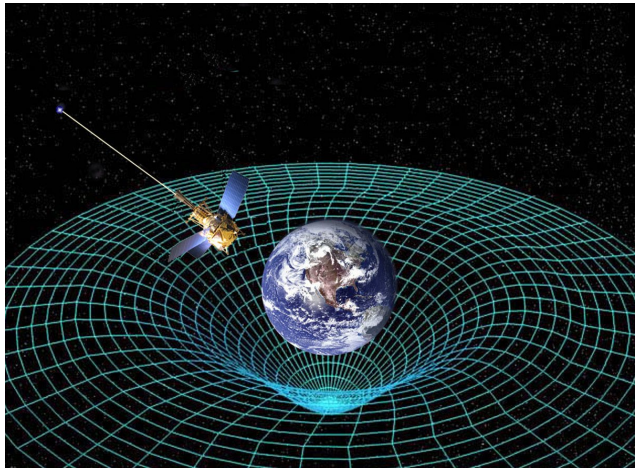
- Factory: leads to higher cost for producing the same shirt
- CPU: leads to **high energy use** for executing the same instructions
  - Functional units still use energy while slacking off

- So, 3 workers or 1 worker per shirt? It depends!
  - Gaming CPU where response time is important → 3 workers
  - Mobile CPU where energy efficiency is important → 1 worker
  - GPU where throughput is important → 1 worker
    - It is not about how fast each pixel or data item is processed
    - It is about how many items can be processed per second

Application Pull
&
Technology Push

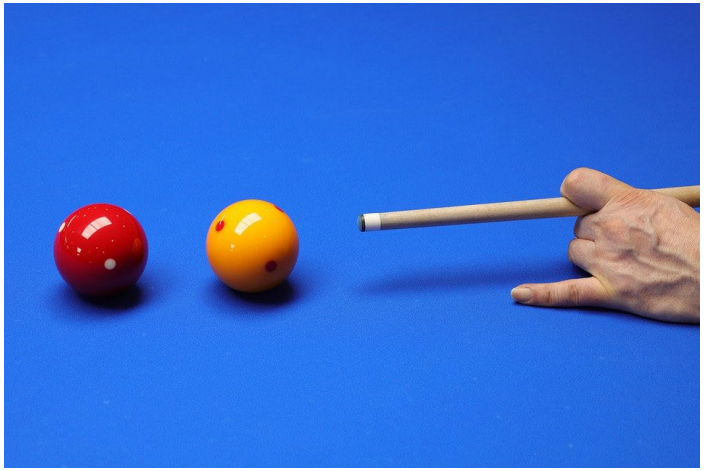
Two Forces on Computer Architecture

1. Application pull



- Market forces pull architecture towards popular applications

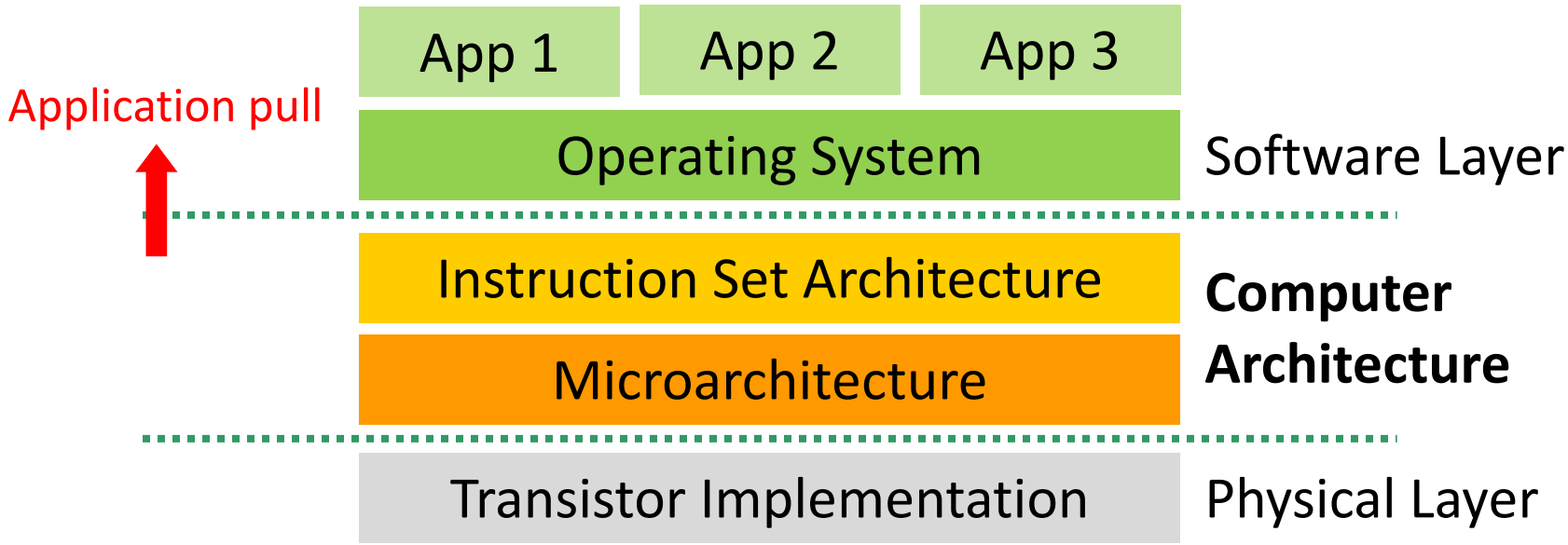
2. Technology push



- Advances in silicon technology push architecture to change

Application Pull

- Different applications pull in different directions



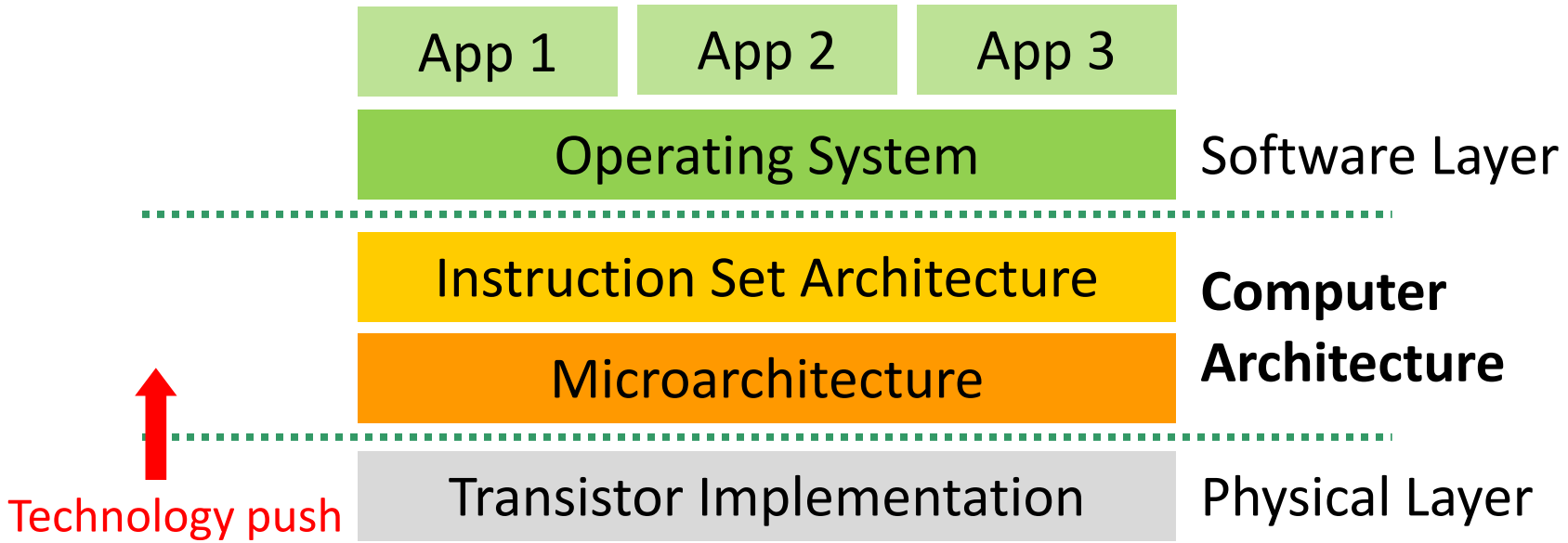
- Real-time app (e.g. Game): *Short latency*
- Server app: *High throughput*
- Mobile app: *High energy-efficiency* (battery life)
- Mission critical app: *High reliability*

Application Pull

- Some goals can be incompatible
  - E.g. reliability and energy-efficiency are conflicting
- Even when sharing a goal, apps have unique needs
  - Scientific apps need lots of *floating point units* to go fast
  - Database apps need lots of *memory cache* to go fast
- An architecture is a compromise among all the apps
  - When app achieves market critical mass, designs diverge (Server chips / Desktop chips / Mobile chips)
  - Sometimes even ISAs diverge (GPUs and TPUs)

Technology Push

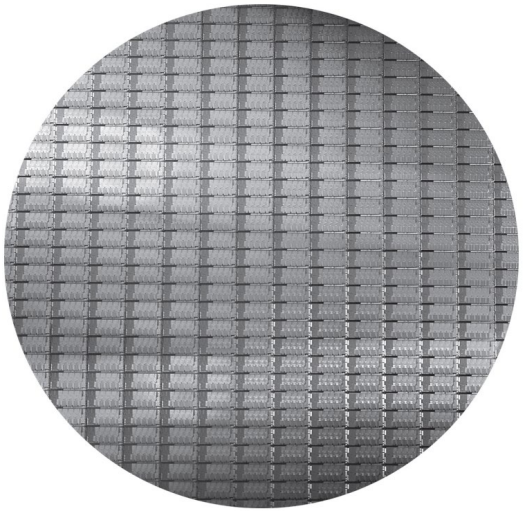
- Trends in technology pushes architecture too



  - Trends can be *advances* in technology
  - Trends can be *constraints* technology couldn't overcome
- "Technology" in CPU design refers to the physical layer
  - Manufacturing technology used for transistor implementation

Scope of Class

- Physical layer is beyond the scope of the class



- The class will focus on instruction set architecture and processor organization